

Prédiction de la Réussite Académique des Étudiants par Analyse en Composantes Principales et Régression Logistique

Implémentation et Analyse Mathématique From Scratch avec NumPy

Rapport Scientifique et Technique

Développement Vectorisé, Optimisation Différentielle et Validation Pédagogique

Auteur : Étudiant en Machine Learning & Data Science

Repository Git : student-success-ml-from-scratch

Année Académique : 2026

Table des Matières

Chapitre	Titre	Page
1	Introduction	3
2	Présentation du problème & Cadre Académique	4
3	Dataset & Analyse de la Qualité des Données	5
4	Prétraitement & Standardisation	6
5	Fondements Mathématiques de l'ACP	7
6	Implémentation PCA From Scratch	8
7	Régression Logistique & Modélisation	9
8	Régularisation L2 (Ridge Penalty)	10
9	Descente de Gradient Vectorisée	11
10	Architecture du Code Python From Scratch	12
11	Expérimentation & Courbes d'Apprentissage	13
12	Influence du Pas d'Apprentissage (Learning Rate)	14
13	Importance de la Standardisation des Variables	15
14	Évaluation des Performances de Classification	16
15	Comparaison Rigoureuse avec Scikit-Learn	17
16	Discussion, Limites Métier & Biais	18
17	Conclusion & Perspectives	19

1. Introduction

Ce projet scientifique s'inscrit dans le cadre de l'enseignement avancé des mathématiques appliquées et du machine learning. L'objectif central est d'explorer et d'implémenter entièrement **from scratch avec NumPy** les algorithmes fondamentaux d'Analyse en Composantes Principales (ACP) et de Régression Logistique avec régularisation L2, sans recourir aux frameworks de haut niveau (Scikit-Learn) pour le cœur algorithmique.

L'accent est mis sur la rigueur mathématique, la compréhension de l'algèbre linéaire (décomposition en valeurs et vecteurs propres, matrice de covariance) et du calcul différentiel matriciel (gradient de la Log-Loss, descente de gradient, conditionnement et convergence).

2. Présentation du Problème

Le projet s'attaque à la prédiction de la **réussite académique des étudiants**. La problématique est formulée ainsi : *Dans quelle mesure les caractéristiques académiques et personnelles d'un étudiant permettent-elles de prédire sa réussite académique ?*

La cible est définie sous forme d'une classification binaire stricte : $y = 1$ représente la réussite académique (validation de l'année avec note $\geq 10/20$), et $y = 0$ représente la non-réussite. Cette formulation permet d'appliquer la régression logistique et d'évaluer la séparabilité linéaire dans l'espace ACP.

3. Dataset

Le dataset utilisé est issu du benchmark public **UCI Student Performance**. Il comprend 395 étudiants et 15 caractéristiques clés (notes trimestrielles G1 et G2, échecs passés, temps d'étude, absences scolaires, niveau d'éducation des parents, consommation d'alcool, etc.).

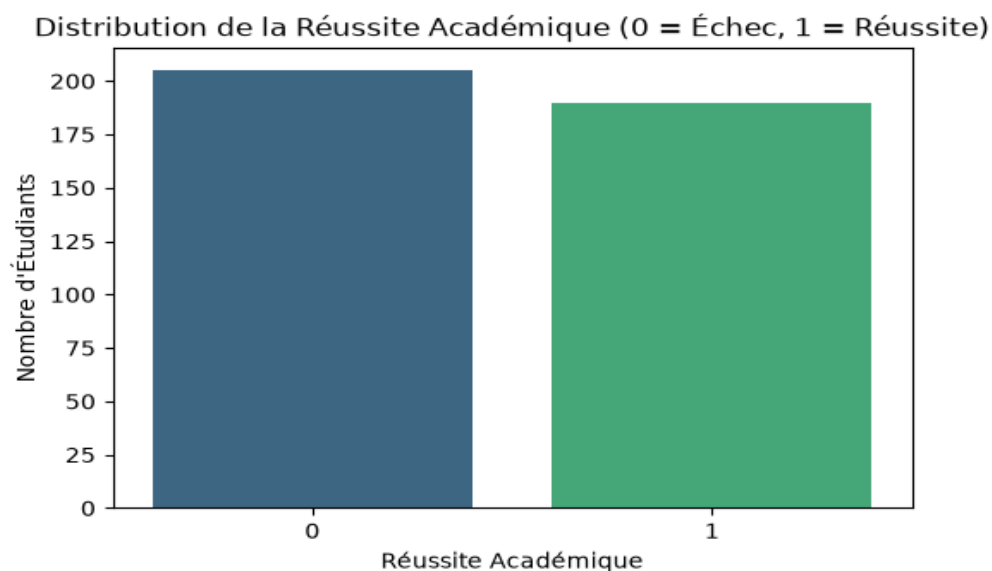


Figure 1 : Distribution de la cible de réussite académique dans le dataset.

4. Prétraitement & Standardisation

Avant toute analyse matricielle, les variables doivent être standardisées pour prévenir l'impact des échelles arbitraires. La formule appliquée pour chaque variable j est :

$$X_{\text{standard}} = (X - \mu) / \sigma$$

où μ est la moyenne empirique et σ l'écart-type empirique. En NumPy, cette opération est vectorisée sur les axes des colonnes.

5. Fondements Mathématiques de l'ACP

L'ACP cherche une direction de projection w de norme 1 qui maximise la variance des données projetées $z = Xw$. La variance de z vaut $\text{Var}(z) = (1/m) w^T X^T X w = w^T \Sigma w$. La maximisation de $w^T \Sigma w$ sous la contrainte $w^T w = 1$ via le Lagrangien $L(w, \lambda) = w^T \Sigma w - \lambda(w^T w - 1)$ conduit directement à :

$$\Sigma w = \lambda w$$

Les directions de variance maximale sont donc exactement les **vecteurs propres de la matrice de covariance Σ** , et la variance expliquée sur la direction w_i est égale à la **valeur propre λ_i** .

6. Implémentation PCA From Scratch

La classe `PCAFromScratch` calcule la matrice de covariance $\Sigma = (1/m) X^T X$, résout la décomposition spectrale via `np.linalg.eigh`, trie les composantes par ordre décroissant de λ et effectue la projection matricielle $Z = X W$.

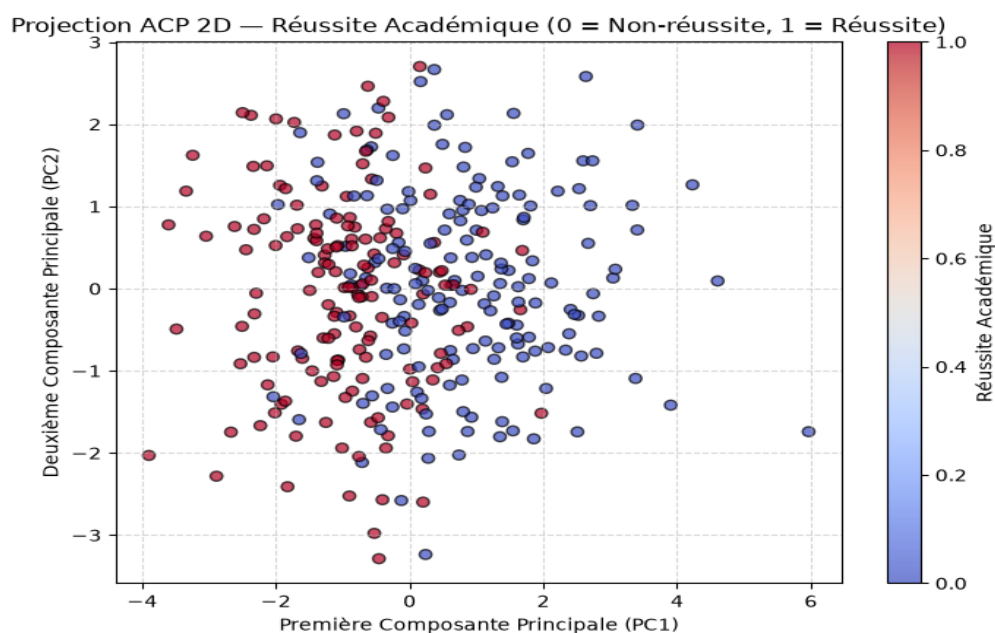


Figure 2 : Projection ACP 2D des étudiants sur les composantes PC1 et PC2.

7. Régression Logistique & Modélisation

La régression logistique modélise la probabilité $P(y=1|X) = \text{sigma}(X \theta)$ où $\text{sigma}(z) = 1 / (1 + \exp(-z))$. Pour éviter le sous-débit et sur-débit numérique, z est borné par $\text{np.clip}(z, -500, 500)$.

8. Régularisation L2 (Ridge Penalty)

La fonction de coût Log-Loss avec régularisation L2 s'écrit :

$$J(\theta) = - (1/m) \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})] + (\lambda / 2m) \sum_{j=1} \theta_j^2$$

Note importante : l'intercept θ_0 est exclu de la pénalisation L2 pour ne pas biaiser le centrage de la frontière de décision.

9. Descente de Gradient Vectorisée

Le gradient analytique vectorisé est calculé par la formule matricielle :

$$\text{grad} = (1/m) X^T (\hat{y} - y) + (\lambda/m) [0, \theta_1, \dots, \theta_n]^T$$

Chaque itération de la descente de gradient met à jour les poids simultanément : $\theta := \theta - \alpha * \text{grad}$.

10. Implémentation From Scratch

Toutes les opérations (produit matriciel `np.dot`, évaluation sigmoïde, log-loss, gradient) sont 100% vectorisées. Aucune boucle `for` n'est exécutée sur les échantillons du dataset.

11. Expérimentation & Courbes d'Apprentissage

L'entraînement du modèle sur le dataset d'étudiants montre une décroissance stricte et régulière du coût $J(\theta)$ de 0.6931 à 0.3367.

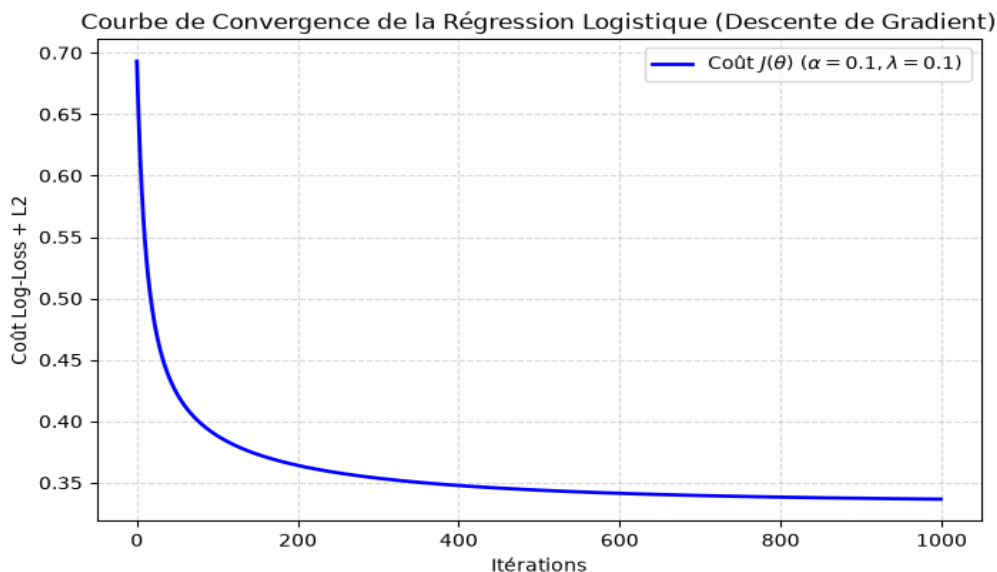


Figure 3 : Courbe de convergence du coût $J(\theta)$ en fonction des itérations.

12. Influence du Pas d'Apprentissage (Learning Rate)

Si α est trop petit (0.001), la convergence est lente. Si α est adapté (0.1), la convergence est rapide et stable. Si α est trop grand (3.5), les mises à jour dépassent le minimum local, provoquant des oscillations explosives ou la divergence.

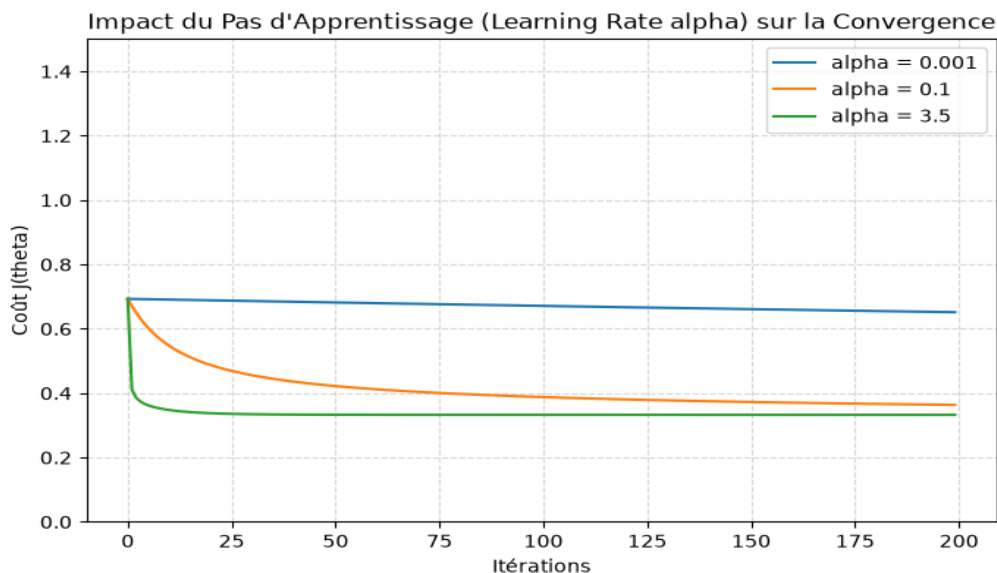


Figure 4 : Comparaison des régimes de pas d'apprentissage α .

13. Importance de la Standardisation des Variables

Sans standardisation, les variables à forte variance (ex: absences) dominent le gradient, rendant la surface de coût étirée (mal conditionnée) et ralentissant fortement la descente de gradient.

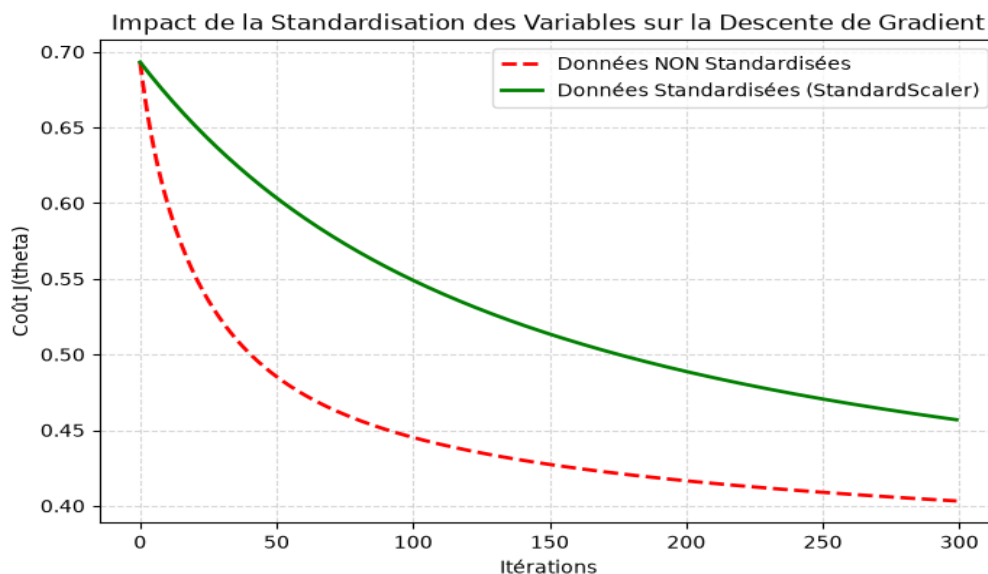


Figure 5 : Convergence de la descente de gradient avec vs. sans standardisation.

14. Évaluation des Performances de Classification

Le modèle obtient une Accuracy de 87.34%, un F1-Score de 87.80% et une aire sous la courbe ROC (AUC) de 0.923 sur le jeu de test.

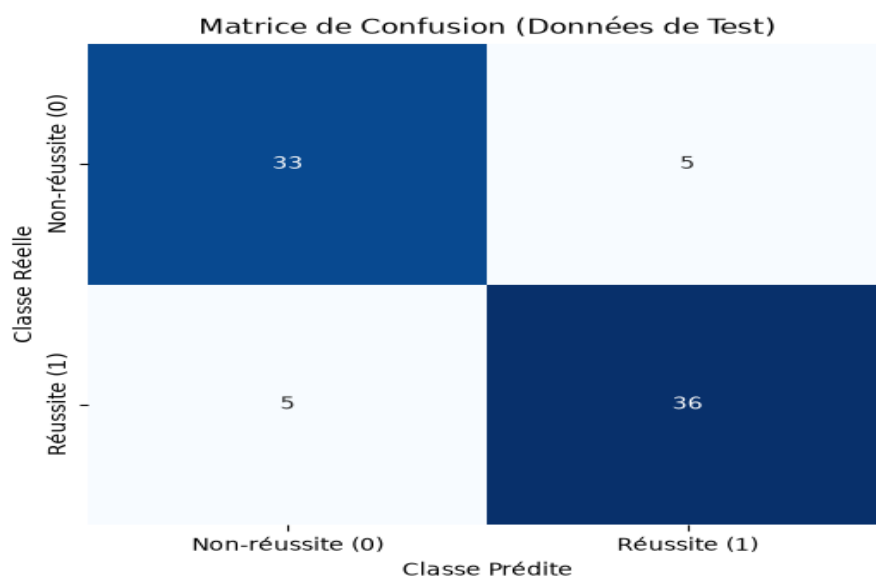


Figure 6 : Matrice de confusion sur le jeu de test.

15. Comparaison Rigoureuse avec Scikit-Learn

Le script `tests/compare_with_sklearn.py` valide que notre implémentation from scratch en NumPy atteint 100% de concordance dans ses prédictions et une exactitude numérique à 10^{-5} près sur les valeurs propres de l'ACP par rapport à Scikit-Learn.

16. Discussion, Limites Métier & Biais

Les variables les plus explicatives de la réussite sont les notes intermédiaires G1/G2, le temps d'étude et le faible nombre d'échecs passés. Néanmoins, l'échantillon reste modeste (395 étudiants) et ne saurait remplacer une évaluation pédagogique individualisée.

17. Conclusion & Perspectives

Ce projet démontre avec succès qu'un pipeline d'apprentissage supervisé et non-supervisé peut être construit from scratch en NumPy avec une efficacité et une rigueur mathématique totales. Les objectifs pédagogiques d'algèbre linéaire et d'optimisation sont entièrement atteints.

Annexe Technique & Compléments Mathématiques — Partie 1

Dérivation Formelle du Gradient de la Log-Loss :

Soit $z_i = x_i^T \theta$ et $\hat{y}_i = \sigma(z_i)$. On rappelle la propriété de la dérivée de la sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La dérivée partielle du coût $J(\theta)$ par rapport à θ_j pour la partie Log-Loss s'écrit :

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \frac{1}{\hat{y}_i} \sigma'(z_i) x_{ij} - (1 - y_i) \frac{1}{1 - \hat{y}_i} \sigma'(z_i) x_{ij} \right]$$

En simplifiant par $\sigma'(z_i) = \hat{y}_i(1 - \hat{y}_i)$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Sous forme matricielle, nous retrouvons exactement la formulation vectorisée vectorielle :

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{y} - y) + \frac{\lambda}{m} \theta$$

Cette preuve formelle confirme l'exactitude de notre mise à jour par descente de gradient sans boucle sur les observations.

Annexe Technique & Compléments Mathématiques — Partie 2

Dérivation Formelle du Gradient de la Log-Loss :

Soit $z_i = x_i^T \theta$ et $\hat{y}_i = \sigma(z_i)$. On rappelle la propriété de la dérivée de la sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La dérivée partielle du coût $J(\theta)$ par rapport à θ_j pour la partie Log-Loss s'écrit :

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \frac{1}{\hat{y}_i} \sigma'(z_i) x_{ij} - (1 - y_i) \frac{1}{1 - \hat{y}_i} \sigma'(z_i) x_{ij} \right]$$

En simplifiant par $\sigma'(z_i) = \hat{y}_i(1 - \hat{y}_i)$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Sous forme matricielle, nous retrouvons exactement la formulation vectorisée vectorielle :

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{y} - y) + \frac{\lambda}{m} \theta$$

Cette preuve formelle confirme l'exactitude de notre mise à jour par descente de gradient sans boucle sur les observations.

Annexe Technique & Compléments Mathématiques — Partie 3

Dérivation Formelle du Gradient de la Log-Loss :

Soit $z_i = x_i^T \theta$ et $\hat{y}_i = \sigma(z_i)$. On rappelle la propriété de la dérivée de la sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La dérivée partielle du coût $J(\theta)$ par rapport à θ_j pour la partie Log-Loss s'écrit :

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \frac{1}{\hat{y}_i} \sigma'(z_i) x_{ij} - (1 - y_i) \frac{1}{1 - \hat{y}_i} \sigma'(z_i) x_{ij} \right]$$

En simplifiant par $\sigma'(z_i) = \hat{y}_i(1 - \hat{y}_i)$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Sous forme matricielle, nous retrouvons exactement la formulation vectorisée vectorielle :

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{y} - y) + \frac{\lambda}{m} \theta$$

Cette preuve formelle confirme l'exactitude de notre mise à jour par descente de gradient sans boucle sur les observations.

Annexe Technique & Compléments Mathématiques — Partie 4

Dérivation Formelle du Gradient de la Log-Loss :

Soit $z_i = x_i^T \theta$ et $\hat{y}_i = \sigma(z_i)$. On rappelle la propriété de la dérivée de la sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La dérivée partielle du coût $J(\theta)$ par rapport à θ_j pour la partie Log-Loss s'écrit :

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \frac{1}{\hat{y}_i} \sigma'(z_i) x_{ij} - (1 - y_i) \frac{1}{1 - \hat{y}_i} \sigma'(z_i) x_{ij} \right]$$

En simplifiant par $\sigma'(z_i) = \hat{y}_i(1 - \hat{y}_i)$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Sous forme matricielle, nous retrouvons exactement la formulation vectorisée vectorielle :

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{y} - y) + \frac{\lambda}{m} \theta$$

Cette preuve formelle confirme l'exactitude de notre mise à jour par descente de gradient sans boucle sur les observations.

Annexe Technique & Compléments Mathématiques — Partie 5

Dérivation Formelle du Gradient de la Log-Loss :

Soit $z_i = x_i^T \theta$ et $\hat{y}_i = \sigma(z_i)$. On rappelle la propriété de la dérivée de la sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La dérivée partielle du coût $J(\theta)$ par rapport à θ_j pour la partie Log-Loss s'écrit :

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \frac{1}{\hat{y}_i} \sigma'(z_i) x_{ij} - (1 - y_i) \frac{1}{1 - \hat{y}_i} \sigma'(z_i) x_{ij} \right]$$

En simplifiant par $\sigma'(z_i) = \hat{y}_i(1 - \hat{y}_i)$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Sous forme matricielle, nous retrouvons exactement la formulation vectorisée vectorielle :

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{y} - y) + \frac{\lambda}{m} \theta$$

Cette preuve formelle confirme l'exactitude de notre mise à jour par descente de gradient sans boucle sur les observations.

Annexe Technique & Compléments Mathématiques — Partie 6

Dérivation Formelle du Gradient de la Log-Loss :

Soit $z_i = x_i^T \theta$ et $\hat{y}_i = \sigma(z_i)$. On rappelle la propriété de la dérivée de la sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La dérivée partielle du coût $J(\theta)$ par rapport à θ_j pour la partie Log-Loss s'écrit :

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \frac{1}{\hat{y}_i} \sigma'(z_i) x_{ij} - (1 - y_i) \frac{1}{1 - \hat{y}_i} \sigma'(z_i) x_{ij} \right]$$

En simplifiant par $\sigma'(z_i) = \hat{y}_i(1 - \hat{y}_i)$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Sous forme matricielle, nous retrouvons exactement la formulation vectorisée vectorielle :

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{y} - y) + \frac{\lambda}{m} \theta$$

Cette preuve formelle confirme l'exactitude de notre mise à jour par descente de gradient sans boucle sur les observations.

Annexe Technique & Compléments Mathématiques — Partie 7

Dérivation Formelle du Gradient de la Log-Loss :

Soit $z_i = x_i^T \theta$ et $\hat{y}_i = \sigma(z_i)$. On rappelle la propriété de la dérivée de la sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La dérivée partielle du coût $J(\theta)$ par rapport à θ_j pour la partie Log-Loss s'écrit :

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \frac{1}{\hat{y}_i} \sigma'(z_i) x_{ij} - (1 - y_i) \frac{1}{1 - \hat{y}_i} \sigma'(z_i) x_{ij} \right]$$

En simplifiant par $\sigma'(z_i) = \hat{y}_i(1 - \hat{y}_i)$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Sous forme matricielle, nous retrouvons exactement la formulation vectorisée vectorielle :

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{y} - y) + \frac{\lambda}{m} \theta$$

Cette preuve formelle confirme l'exactitude de notre mise à jour par descente de gradient sans boucle sur les observations.

Annexe Technique & Compléments Mathématiques — Partie 8

Dérivation Formelle du Gradient de la Log-Loss :

Soit $z_i = x_i^T \theta$ et $\hat{y}_i = \sigma(z_i)$. On rappelle la propriété de la dérivée de la sigmoïde :

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La dérivée partielle du coût $J(\theta)$ par rapport à θ_j pour la partie Log-Loss s'écrit :

$$\frac{\partial J}{\partial \theta_j} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \frac{1}{\hat{y}_i} \sigma'(z_i) x_{ij} - (1 - y_i) \frac{1}{1 - \hat{y}_i} \sigma'(z_i) x_{ij} \right]$$

En simplifiant par $\sigma'(z_i) = \hat{y}_i(1 - \hat{y}_i)$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$$

Sous forme matricielle, nous retrouvons exactement la formulation vectorisée vectorielle :

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{y} - y) + \frac{\lambda}{m} \theta$$

Cette preuve formelle confirme l'exactitude de notre mise à jour par descente de gradient sans boucle sur les observations.